Backend Explanation

The back-end folder is built using NestJS with TypeScript. It works as the REST API for the Click2Book online ticket booking system.

Folder Preparation

I prepared the backend in a modular way. Each major feature has its own module inside src/modules, such as:

- auth
- customer
- admin
- provider
- vehicle
- seat
- route
- schedule
- trip
- booking
- payment
- cancellation
- refund
- review
- support-request
- report

Each module follows the same structure:

- controller.ts handles API routes.
- service.ts contains business logic.
- repository.ts manages data operations.
- dto.ts validates input data.
- interface.ts defines data structure.
- enum.ts stores fixed status/type values.

Main Configuration

The backend starts from src/main.ts. In this file, I configured:

- Global API prefix: /api
- CORS
- Input validation using ValidationPipe
- Global exception handling
- Swagger API documentation

The root file src/app.module.ts connects all feature modules together.

Common Features

The src/common folder contains reusable code:

- Standard API response format
- ID generation utility
- Role-based access control
- Global error handling
- Common role enums

Role-Based Access

The backend uses an x-role request header for access control. Roles include:

- CUSTOMER
- ADMIN
- PROVIDER
- SUPPORT

APIs are protected using guards and role decorators.

Example Flow

In the booking module:

1. The controller receives the booking request.
2. The service checks trip, seat, and vehicle availability.
3. The repository stores booking data.
4. The response is returned in a standard format.

Conclusion

Overall, the backend is prepared as a clean, modular, and maintainable NestJS API. It supports ticket booking features, role-based access, validation, error handling, and Swagger documentation for easy API testing.